Universidad Mariano Galvez de Guatemala

Curso: programación lll

Tema: parcial ll

Docente: Inge. David Alvarez

Alumno: Carlos Daniel Castro Caceres

```
521 #include<iostream>                    Carlos Daniel Castro Cáceres
522 #include <stdlib.h>
492
493 Usando el espacio de nombres STD;
 1
 2 struct NodoPila{
 3     int num;
 4     NodoPila *sig;
 5 };
 6
 7 struct NodoCola{
 8     int num;
 9     NodoCola *sig;
10 };
11
12 struct NodoLista{
13     int num;
14     NodoLista *sig;
15 };
16
517 struct NodoDoble{
518     int num;
519     Tipo de cuerda;
520     int orden;
521     NodoDoble *sig;
522     NodoDoble *hormiga;
523 };
517
518 struct NodoArbol{
```

```
518 struct NodoArbol{
519     int num;
520     NodoArbol *izq;
521     NodoArbol *der;
523 };
522
492 NodoPila *tope = NULL;
493 NodoCola *frente = NULL, *fin = NULL;
 1  NodoLista *lista = NULL;
 2  NodoDoble *hini = NULL, *hfin = NULL;
 3  NodoArbol *raiz = NULL;
 5
 4  int orden = 1;
 7  int pares = 0;
 6  int impares = 0;
 8  int negativos = 0;
10  int ceros = 0;
 9  int validos = 0;
12  int mayor = 0;
11  int menor = 0;
13  bool primero = verdadero;
14
15  Pausa del vacío (){
16      cout << "Presione Enter para continuar...";
517     cin. ignorar(1000,'\n');
518     Cin. ¡Haz();
519 }
520
521 Void Push (int num){
```

```
521 Void Push (int num){
522     NodoPila *nuevo = nuevo NodoPila();
523     Nuevo->num = num;
517     nuevo->sig = tope;
518     tope = nuevo;
519 }
520
521 Hollow Encolar(int num){
523     NodoCola *nuevo = nuevo NodoCola();
522     Nuevo->num = num;
492     nuevo->sig = NULL;
493
  1     if(frente == NULL){
  2         frente = nuevo;
  3         fin = nuevo;
  5     }si no,{
  4         fin->sig = nuevo;
  7         fin = nuevo;
  6     }
  8 }
 10
  9 void insertarLista(int num){
 12     NodoLista *nuevo = nuevo NodoLista();
 11     Nuevo->num = num;
 13     nuevo->sig = NULL;
 14
 15     if(lista == NULL){
 16         lista = nuevo;
517     }si no,{
```

```
517         while(aux->sig != NULL){
518             aux = aux->sig;
519         }
520         aux->sig = nuevo;
521     }
522 }
523
517  void historial(int num, tipo de cadena){
518     NodoDoble *nuevo = nuevo NodoDoble();
520     Nuevo->num = num;
519     Nuevo->tipo = tipo;
521     Nuevo->orden = orden++;
523     nuevo->sig = NULL;
492     nuevo->ant = hfin;
522
  1     if(hini == NULL){
493         hini = nuevo;
  2         hfin = nuevo;
  3     }si no,{
  4         hfin->sig = nuevo;
  5         hfin = nuevo;
  7     }
  6 }
  8
 10 void insertarBST(NodoArbol *&r, int num){
  9     if(r == NULL){
 12         r = nuevo NodoArbol();
 11         r->num = num;
 13         r->izq = NULL;
```

```
13              r->izq = NULL;
14              r->der = NULL;
15          }si no, if(num < r->num){
16              insertarBST(r->izq, num);
517         }si no, if(num > r->num){
518             insertarBST(r->der, num);
519         }
520     }
521
522     void inOrden(NodoArbol *r){
523         if(r != NULL){
517             inOrden(r->izq);
520             Cout << r->num << " ";
518             en Orden(r->der);
519         }
521     }
523
492     void preOrden(NodoArbol *r){
522         if(r != NULL){
  1             Cout << r->num << " ";
493             preOrden(r->izq);
  2             preOrden(r->der);
  3         }
  4     }
  5
  7     void postOrden(NodoArbol *r){
  6         if(r != NULL){
  8             postOrden(r->izq);
 10             postOrden(r->der);
```

```
 10             postOrden(r->der);
  9             Cout << r->num << " ";
 12         }
 11     }
 13
 14     bool buscarBST(NodoArbol *r, int num){
 15         if(r == NULL) return false;
 16         if(r->num == num) devolver true;
517
518         if(num < r->num)
519             return buscarBST(r->izq, num);
520         si no,
521             return buscarBST(r->der, num);
522     }
523
517     void mostrarPila(){
520         NodoPila *aux = tope;
518
519         if(aux == NULL){
523             cout << "Pila vacia"";
521             Regreso;
492         }
522
  1         while(aux != NULL){
493             cout << aux->num << endl;
  2             aux = aux->sig;
  3         }
  4     }
  5
```

```
 5
 7  vacuo mostrarCola(){
 6      NodoCola *aux = frente;
 8
10      if(aux == NULL){
 9          cout << "Cola vacia"";
12          Regreso;
11      }
13
14      while(aux != NULL){
15          cout << aux->num << endl;
16          aux = aux->sig;
518     }
517 }
519
520 void mostrarLista(){
521     NodoLista *aux = lista;
522
523     if(aux == NULL){
517         cout << "Lista vacia"";
520         Regreso;
518     }
519
523     while(aux != NULL){
521         Cout << aux->num << " ";
492         aux = aux->sig;
522     }
 1
493     ¡Sin << fin!;
```

```
493     ¡Sin << fin!;
  2 }
  3
  4 void mostrarHistorial(){
  5     NodoDoble *aux = hini;
  7
  6     if(aux == NULL){
  8         cout << "Sin historial"";
 10         Regreso;
  9     }
 12
 11     while(aux != NULL){
 13         cout << aux->orden << ". ";
 14         Cout << aux->num << " -> ";
 15         Cout << aux->tipo << endl;
 16         aux = aux->sig;
517     }
518 }
519
520 void liberarArbol(NodoArbol *&r){
521     if(r != NULL){
522         liberarArbol(r->IZQ);
523         liberarArbol(r->der);
517         Eliminar r;
520         r = NULL;
518     }
519 }
523
521 void construirBST(){
```

```
521 void construirBST(){
492     liberarArbol(raiz);
522     raiz = NULL;
  1
493     NodoPila *a = tope;
  2     while(a != NULL){
  3         insertarBST(raiz, a->num);
  7         A = A->sig;
  5     }
  4
  6     NodoCola *b = frente;
  8     while(b != NULL){
 10         insertarBST(raiz, b->num);
  9         b = b->sig;
 12     }
 11
 13     if(raiz == NULL){
 14         cout << "No hay datos positivos"";
 15         Regreso;
 16     }
517
518     cout << "InOrden: ";
519     inOrden(raiz);
520     ¡Sin << fin!;
521
522     cout << "PreOrden:";
523     preOrden(raiz);
517     ¡Sin << fin!;
520
```

```
520
518     cout << "PostOrden: ";
519     postOrden(raiz);
523     ¡Sin << fin!;
521 }
492
522 void eliminarNegativos(){
  1     NodoLista *aux = lista;
493     NodoLista *ant = NULL;
  2     int cont = 0;
  3
  7     cout << "Lista original: ";
  5     mostrarLista();
  4
  6     while(aux != NULL){
  8         if(abs(aux->num) < 10){
 10             NodoLista *temp = aux;
  9
 12             if(aux == lista){
 11                 lista = aux->sig;
 13                 aux = lista;
 14             }si no,{
 15                 ant->sig = aux->sig;
 16                 aux = aux->sig;
517             }
518
519             eliminar la temperatura;
520             cont++;
521         }si no,{
```

```
521         }si no,{
522             Hormiga = auxiliar;
523             aux = aux->sig;
517         }
520     }
518
519     cout << "Lista resultante: ";
523     mostrarLista();
521     cout << "Eliminados: " << continúa << fin;
492 }
522
493 Estadísticas del vacío(){
  1     cout << "Pares positivos: " << pares << endl;
  2     cout << "Impares positivos: " << impares << endl;
  7     cout << "Negativos: " << negativos << final.;
  3     cout << "Ceros rechazados: " << ceros << Endl;
  5     cout << "Total validos: " << validos << endl;
  4
  6     si(validos > 0){
 10         cout << "Alcalde valor: " << alcalde << fin;
  8         cout << "Menor valor: " << menor << fin;
 12     }
  9 }
 13
 11 Reorganizar el vacío (){
 15     liberarArbol(raiz);
 14     raiz = NULL;
 16
517     NodoLista *l1 = NULL;
```

```
517     NodoLista *l1 = NULL;
518     NodoLista *l2 = NULL;
519
520     while(tope != NULL){
521         NodoLista *nuevo = nuevo NodoLista();
522         nuevo->num = tope->num;
523         nuevo->sig = l1;
517         L1 = Nuevo;
520
518         insertarBST(raiz, tope->num);
521
523         NodoPila *temp = tope;
519         tope = tope->sig;
522         eliminar la temperatura;
492     }
  1
493     while(frente != NULL){
  7         NodoLista *nuevo = nuevo NodoLista();
  2         Nuevo->num = frente->num;
  5         nuevo->sig = l2;
  3         L2 = Nuevo;
  4
  6         insertarBST(raiz, frente->num);
 10
  8         NodoCola *temp = frente;
 12         frente = frente->sig;
  9         eliminar la temperatura;
 13     }
 11
```

```
 11
 15        fin = NULL;
 14
 16        cout << "Lista auxiliar pila: ";
517        NodoLista *a = l1;
518        while(a != NULL){
519            Cout << a->num << " ";
520            A = A->sig;
521        }
522
523        ¡Sin << fin!;
517
520        cout << "Lista auxiliar cola: ";
521        NodoLista *b = l2;
518        while(b != NULL){
523            Cout << B->num << " ";
522            b = b->sig;
519        }
492
  1        ¡Sin << fin!;
493
  7        cout << "Nuevo BST InOrden: ";
  2        inOrden(raiz);
  5        ¡Sin << fin!;
  3    }
  4
  6    void liberarTodo(){
 10
  8        while(tope != NULL){
```

```
  8        while(tope != NULL){
 12            NodoPila *temp = tope;
  9            tope = tope->sig;
 13            eliminar la temperatura;
 11        }
 15
 14        while(frente != NULL){
 16            NodoCola *temp = frente;
517            frente = frente->sig;
518            eliminar la temperatura;
519        }
520
521        while(lista != NULL){
522            NodoLista *temp = lista;
523            lista = lista->sig;
517            eliminar la temperatura;
520        }
521
518        while(hini != NULL){
522            NodoDoble *temp = hini;
523            hini = hini->sig;
519            eliminar la temperatura;
492        }
  1
493        liberarArbol(raiz);
  7    }
  2
  5    Int Main(){
  3
```

```
Int Main(){

    int opc, n, num, dato;

    do{
        Cout  << "\n1. Ingresar N números\n";
        Cout << "2. Mostrar pila\n";
        cout << "3. Mostrar cola"";
        cout << "4. Mostrar lista negativos"";
        Cout << "5. Mostrar historial\n";
        cout << "6. Construir y mostrar BST\n";
        cout << "7. Eliminar negativos"";
        cout << "8. Buscar valor en BST\n";
        Cout << "9. Mostrar estadísticas"";
        Cout << "10. Reorganizar estructuras"";
        Cout << "11. Salir"";
        CIN >> OPC;

        Switch (OPC){

        Caso 1:

            cout << "Cantidad: ";
            cin >> n;

            si(n <= 0){
                cout << "Dato invalido\n";
                Pausa;
            }
```

```
            }

            para (int i=1; i<=n; i++){

                cout << "Número" << i << ": ";
                cin >> num;

                if(num == 0){
                    Ceros++;
                }
                si no, si(num >  0 && num % 2 == 0){
                    encolar(num);
                    historial(num, "COLA");
                    Pares++;
                    validos++;
                }
                si no, si(num > 0){
                    push(num);
                    historial(num", "PILA");
                    impares++;
                    validos++;
                }
                si no,{
                    insertarLista(num);
                    historial(num,"LISTA SIMPLE");
                    negativos++;
                    validos++;
                }
```

```
522
523                 if(num != 0){
519                     if(primero){
492                         alcalde = num;
  1                         menor = num;
493                         primero = falso;
  7                     }si no,{
  2                         if(num > mayor) mayor = num;
  5                         if(num < menor) menor = num;
  4                     }
  3                 }
  6             }
  8
 10             Pausa;
  9
 12         Caso 2:
 13             mostrarPila();
 11             Pausa;
 15
 14         Caso 3:
 16             mostrarCola();
517             Pausa;
519
518         Caso 4:
522             mostrarLista();
521             Pausa;
520
523         Caso 5:
517             mostrarHistorial();
```

```
517             mostrarHistorial();
521             Pausa;
520
518         Caso 6:
522             construirBST();
523             Pausa;
519
492         Caso 7:
  1             eliminarNegativos();
493             Pausa;
  7
  2         Caso 8:
  4             cout << "Valor a buscar: ";
  5             Cin >> Dato;
  3
  6             if(buscarBST(raiz,dato))
  8                 cout << "Encontrado"";
 10             si no,
  9                 cout << "No encontrado"";
 12
 13             Pausa;
 11
 15         Caso 9:
 14             Estadísticas();
 16             Pausa;
517
519         Caso 10:
518             Reorganizar();
522             Pausa;
```

```
522             Pausa;
521
520         Caso 11:
523             liberarTodo();
521             cout << "Programa finalizado\n";
517             Pausa;
520
522         Por defecto:
518             cout << "Opcion invalida"";
523         }
519
492         si(opc != 11)
  1             pausa();
493
  7     }while(opc != 11);
  2
  4     retorno 0;
  5 }
```

```
1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
11. Salir
1
Cantidad: 8
Numero 1: 22
Numero 2: 25
Numero 3: -2
Numero 4: 7
Numero 5: -9
Numero 6: 15
Numero 7: 0
Numero 8: 29
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
11. Salir
```

```
11. Salir
2
29
15
7
25
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
11. Salir
3
22
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
11. Salir
4
```

```
4
-2 -9
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
11. Salir
5
1. 22 -> COLA
2. 25 -> PILA
3. -2 -> LISTA SIMPLE
4. 7 -> PILA
5. -9 -> LISTA SIMPLE
6. 15 -> PILA
7. 29 -> PILA
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
```

```
10. Reorganizar estructuras
11. Salir
6
InOrden: 7 15 22 25 29
PreOrden: 29 15 7 25 22
PostOrden: 7 22 25 15 29
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
11. Salir
7
Lista original: -2 -9
Lista resultante: Lista vacia
Eliminados: 2
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
```

```
10. Reorganizar estructuras
11. Salir
8
Valor a buscar: 29
Encontrado
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
11. Salir
9
Pares positivos: 1
Impares positivos: 4
Negativos: 2
Ceros rechazados: 1
Total validos: 7
Mayor valor: 29
Menor valor: -9
Presione Enter para continuar...

1. Ingresar N numeros
2. Mostrar pila
3. Mostrar cola
4. Mostrar lista negativos
5. Mostrar historial
6. Construir y mostrar BST
7. Eliminar negativos
```

```
7. Eliminar negativos
8. Buscar valor en BST
9. Mostrar estadisticas
10. Reorganizar estructuras
11. Salir
10
Lista auxiliar pila: 25 7 15 29
Lista auxiliar cola: 22
Nuevo BST InOrden: 7 15 22 25 29
Presione Enter para continuar...
```